ANDROID APPLICATION MODEL

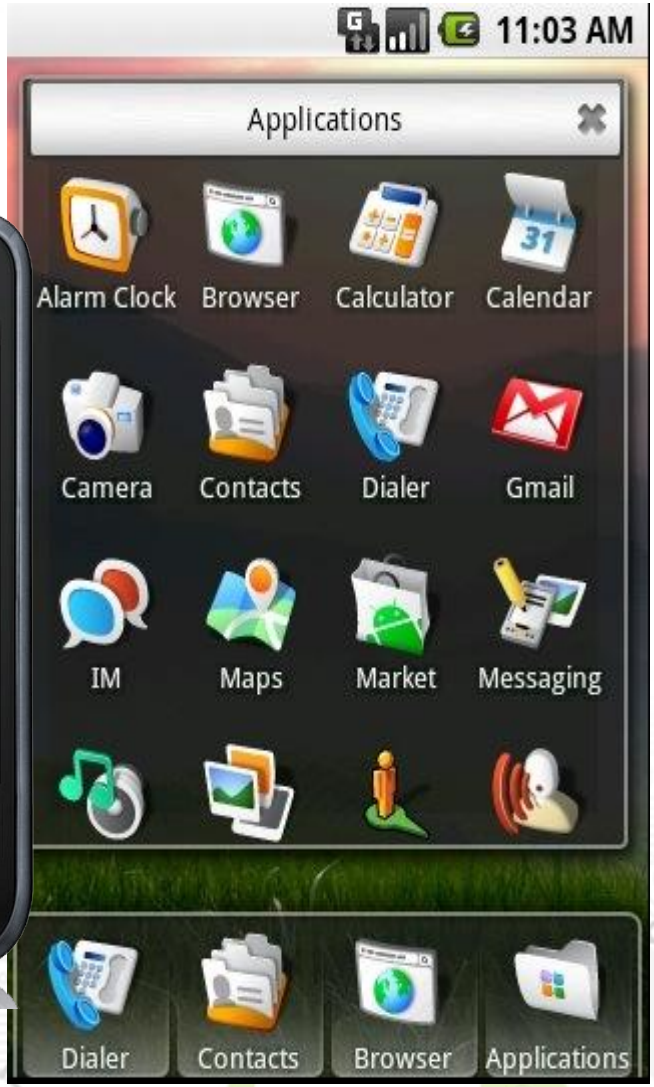
aitrich TECHNOLOGIES

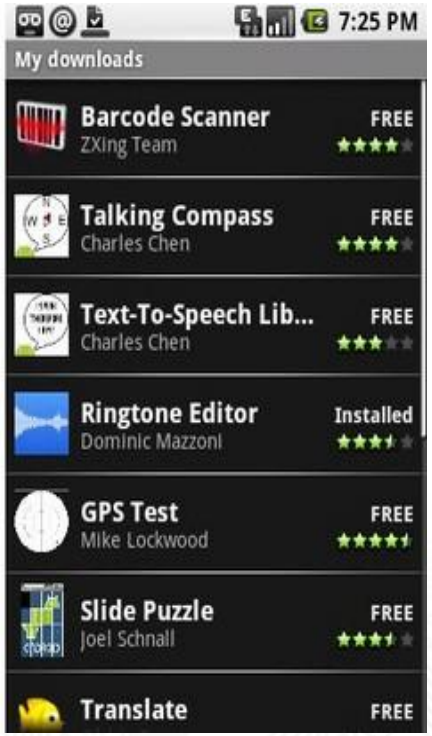
aTP aitrich trained professional

OUTLINE

- Introduction
- Application Fundamentals
- Application Components
- Application Context
- Activating Components
- Deactivating Components
- Task and Processes
- Component Life cycles
- Summary

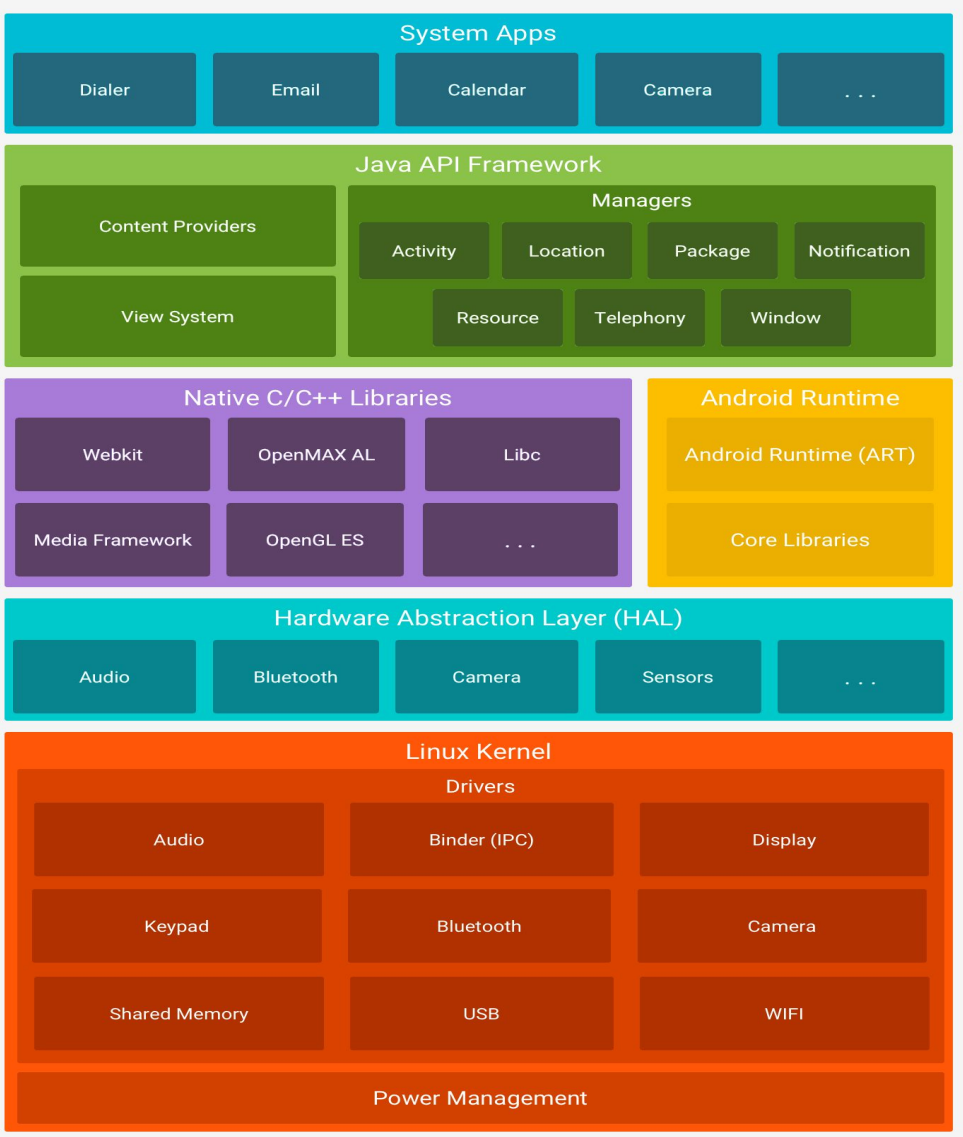
INTRODUCTION



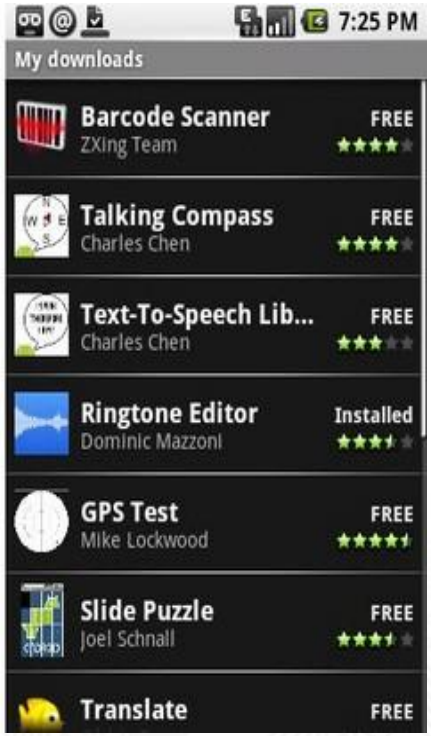


Introduction

- At the top of the stack are the **applications & widget** layer (Built in and user app goes here).
- Applications are programs that can take over the whole screen and interact with the user.
- Widgets(which are sometimes called gadgets),operate only in a small rectangle of the Home screen application.
- End users will see only these programs, blissfully unaware of all the action going on below.



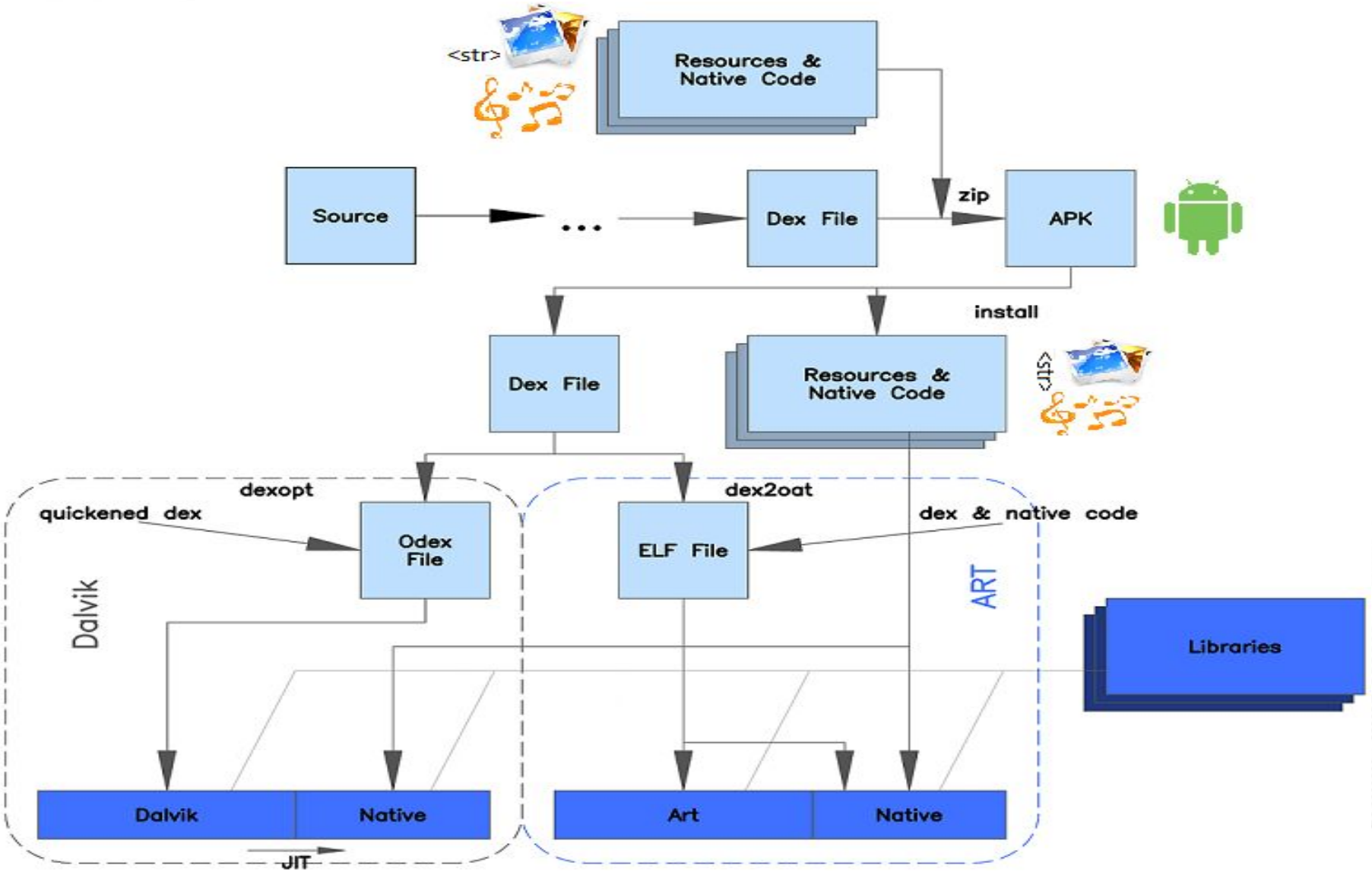
Application



Widget


Beautiful Widgets
12 08
Verdun sur
14°
Vibrate
WiFi
Bluetooth
Plane
Widgets
...and much more !

Producing an Android App



Application Fundamentals

- Android applications are written in Java, but remember it is not Java ME (Mobile Edition). It's just Most of Java SE libraries + Android's own Java libraries.

- The compiled application is bundled by the **aapt** tool into an Android package **.apk**

- Like **.jad** & **.jar** in Java ME, **.apk** files are distributed and installed on Android powered devices.

Application Fundamentals

Basic Facts

- Every Application
  - Is a single Linux process
  - Has its own VM instance
  - Is assigned a unique Linux user ID (the ID is used only by the system and is unknown to the application)

Application Fundamentals

Basic Facts

- A key feature of Android is that applications can "publish“ their functionalities.
- An application can use other functionalities provided by other applications.
- Android applications don't have a single entry point (no main() function, for example). Rather, they have essential ***components*** that the system can instantiate and run as needed.

What an Application Would do

A real world Example

What an Application Would do

Let's say that we want to build a Twitter app. We know that the user should be able to post status updates. We also know the user should be able to see what her friends are up to. Those are basic features. Beyond that, the user should also be able to set her username and password in order to log into her Twitter account. So, now we know we should have these three screens.

Next, we would like this app to work quickly regardless of the network connection or lack thereof.

What an Application Would do

To achieve that, the app has to pull the data from Twitter when it's online
and cache the data locally. That will require a service that runs in the background as
well as a database.
We also know that we'd like this background service to be started when the device is
initially turned on, so by the time the user first uses the app, there's already up-to-date
information on her friends.
So, these are some straightforward requirements.
Android building blocks make it easy
to break them down into conceptual units so that you can work on them independently,
and then easily put them together into a complete package.

What an Application Would do

- Display UI
- Let user navigate from one UI to another
- Provide services
- Act due to cared events
- Store, retrieve , share data
- etc…….

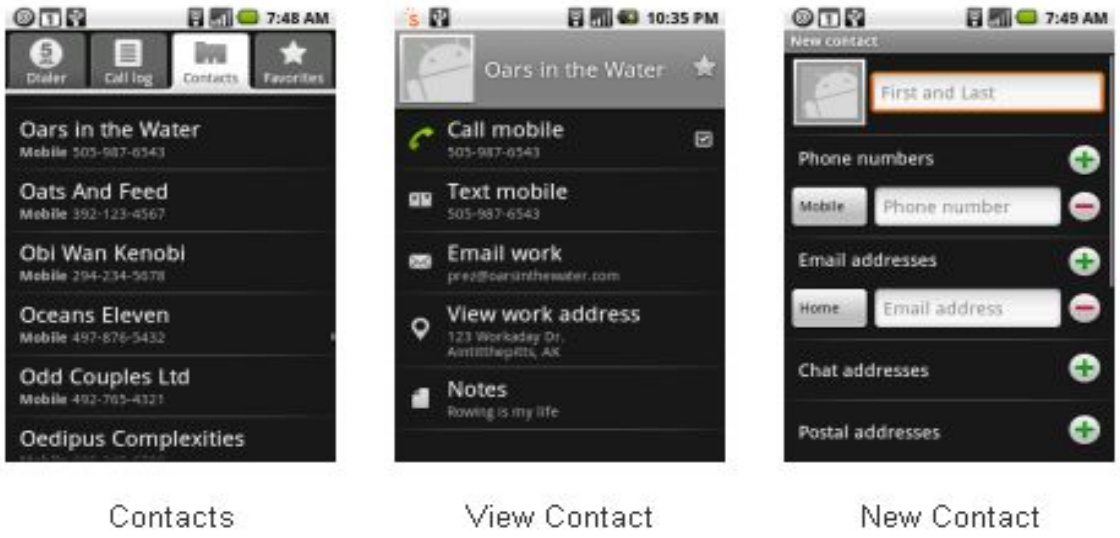
What Makes an Android Application

- Android applications consist of loosely coupled **components** , bound using a project manifest that describes each component and how they interact.
- There are six components that provide the building blocks for android applications:
  - **Activities**
  - **Intents**
  - **Services**
  - **Content Providers**
  - **Broadcast Receivers**
  - **Notifications**

Application Components

Activities

- An **activity** presents the user with a visual UI for a specific functionality.
- Only One is visible and Active, new activities are placed on top.
- Previous screen(activity) is paused & put onto a history Stack.
- E.g. Contacts: 3 activities: View contacts, New Contact, Edit contact.



Activities

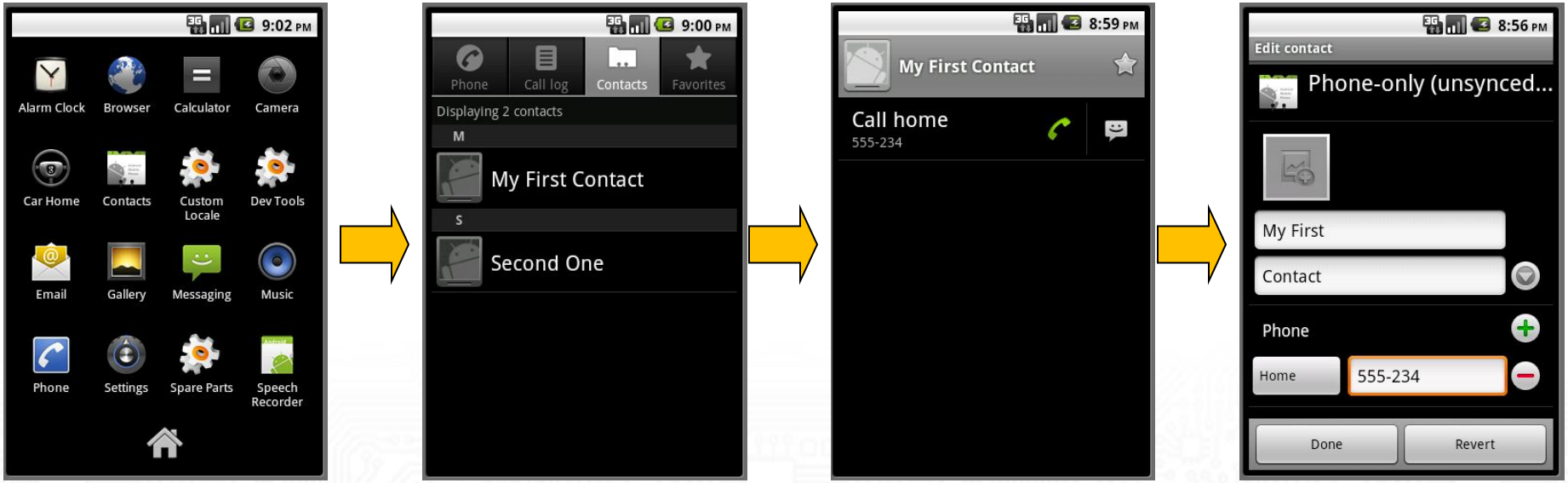
- An application typically consists of several screens:
  - Each activity is given a default window to draw in
  - Moving to the next screen means starting a new activity.
  - An activity may return a result to the previous activity.
  - Each activity has its own life cycle.

- The visual content of the window is provided by a hierarchy of **views** — objects derived from the ***View* class**.

Activities - Views

- Android provides several built-in views:
  - Buttons
  - Menus
  - Text fields
  - ...
- User interacts with activities through the views.
- Views can reacts on user input/events.

Activities


9:02 PM
Alarm Clock
Browser
Calculator
Camera
Car Home
Contacts
Custom Locale
Dev Tools
Email
Gallery
Messaging
Music
Phone
Settings
Spare Parts
Speech Recorder
9:00 PM
Phone
Call log
Contacts
Favorites
Displaying 2 contacts
M
My First Contact
S
Second One
8:59 PM
My First Contact
Call home
555-234
8:56 PM
Edit contact
Phone-only (unsynced...
My First
Contact
Phone
Home
555-234
Done
Revert

Intents

- A request to do something.
- Intents are simple message objects each of which consists of
  - Action to be performed (VIEW/EDIT/PICK/DELETE/DIAL/etc)

```
startActivity(new Intent(Intent.VIEW_ACTION, Uri.parse("geo:47.480843,8.211293"));

startActivity(new Intent(Intent.EDIT_ACTION,Uri.parse("content://contacts/people/1"));
```

- Intents are **asynchronous** messages that are sent among the major building blocks.

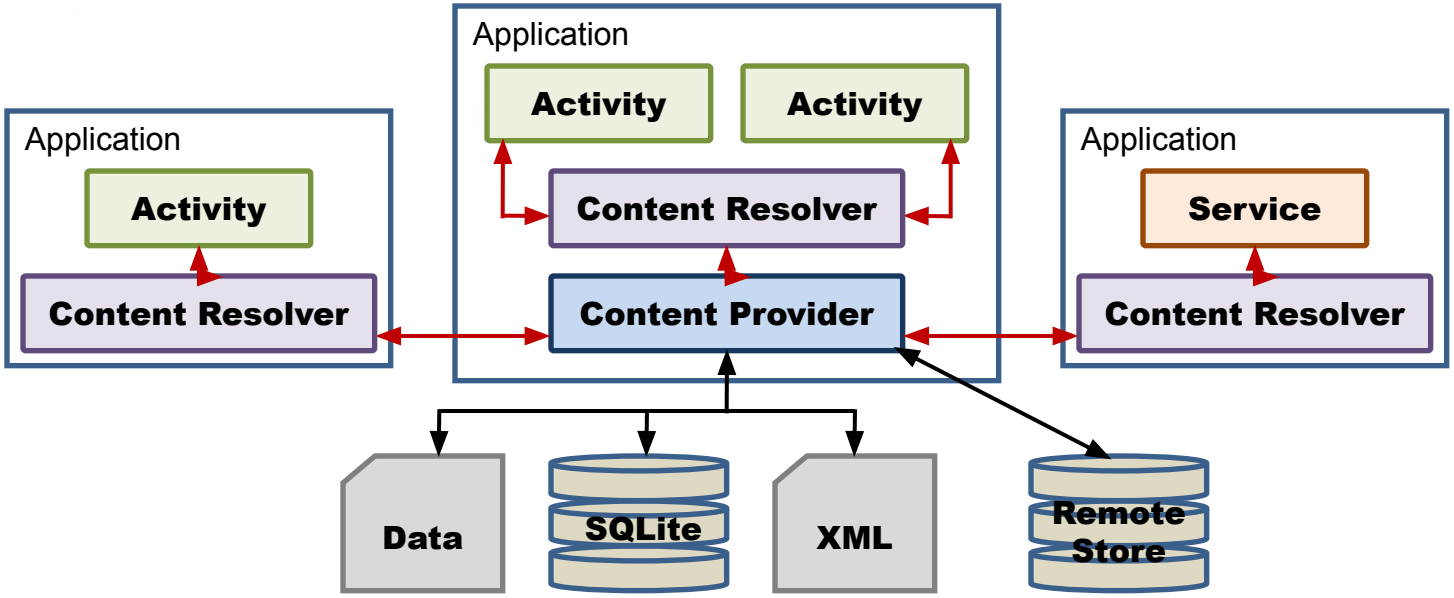
Intents

- Intents in Android fall into two broad categories:
  - Explicit intents - specifies the exact component to be run
  - Implicit intents - where the target component is not named explicitly and the Android system is expected to determine the best component to implement the intent using **intent filters** .
- **Intent Filters** are the way of any component to advertise its own capabilities to the Android system. This is done declaratively in the AndroidManifest.xml file.

Services

- **Services** - A long-running background task.
- A service does not have a visual user interface, but rather runs in the background for an indefinite period time.
  - Example: music player, network download, etc
- How to start a service
  - Start service - A service is "started" when an application component (such as an activity) starts it by calling ***startService()***
  - Bind service - A service is "bound" when an application component binds to it by calling ***bindService().*** (Inter Process communication-IPC)

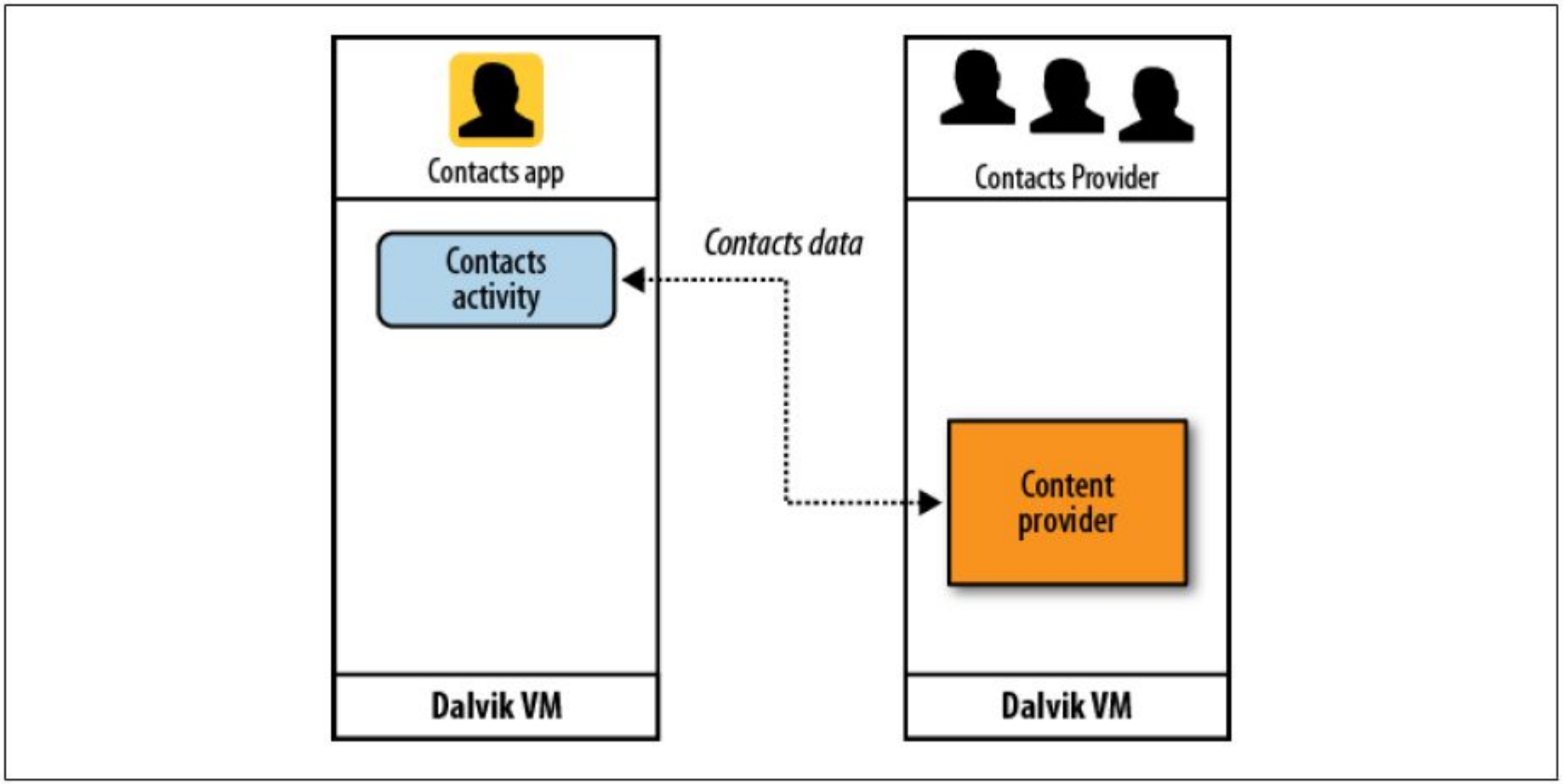
Content Providers



- The only way to share data between android applications. (makes a specific set of the application's data available to other applications)
- Applications "talk" to content provider through a content resolver.
- Any form of data storage can be used like SQLite database, Files, Remote store, Or any other manner that makes sense.

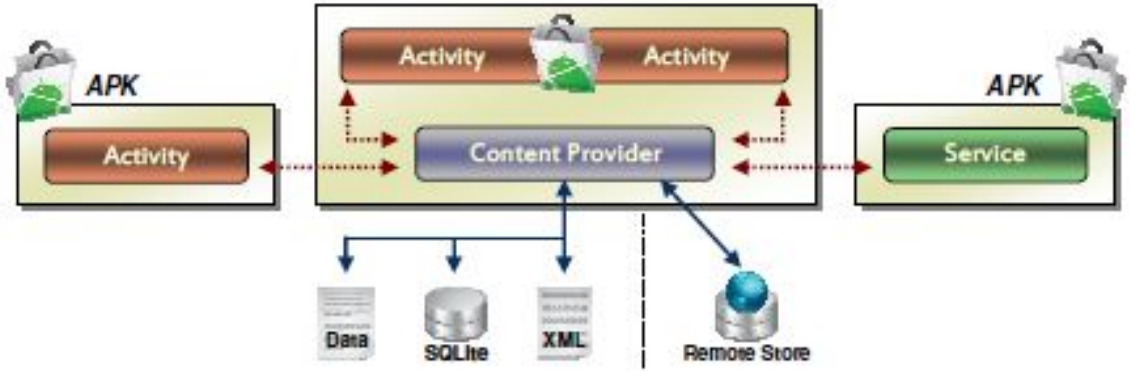
Content Providers

- The Android system uses this mechanism all the time.



Contacts application using Contacts Provider to get the data

Content Providers



- Content providers are relatively simple interfaces, with the standard set of methods to allow access to a data store.
  - Querying, Delete, update, and insert data
- Applications do not call these methods directly.
  - They use a **ContentResolver** object and call its methods instead.
  - A **ContentResolver** can talk to any content provider.
- Android use both the content URI and MIME type as ways to identify the content.

Broadcast Receiver

- A broadcast receiver extends the base class **BroadcastReceiver,** and has only one function
  - To listen for and respond to broadcast announcements.
- Many broadcasts originate from system level
  - Example: announcements that the time zone has changed, that the battery is low, etc.

Broadcast Receiver

- Broadcast receivers do not display a user interface
  - They may start an activity in response to the information they receive
  - Or they may use the **NotificationManager** to alert the user.
- Notifications can get the user's attention in various ways
  - flashing the backlight, vibrating the device, playing a sound, and so on.
  - They typically place a persistent icon in the status bar, which users can open to get the message.

Notifications

- A small icon that appears in the status bar. Users can interact with this icon to receive information.
- Notifications are the strongly-preferred mechanism for alerting the user of something that needs their attention.

AndroidManifest.xml

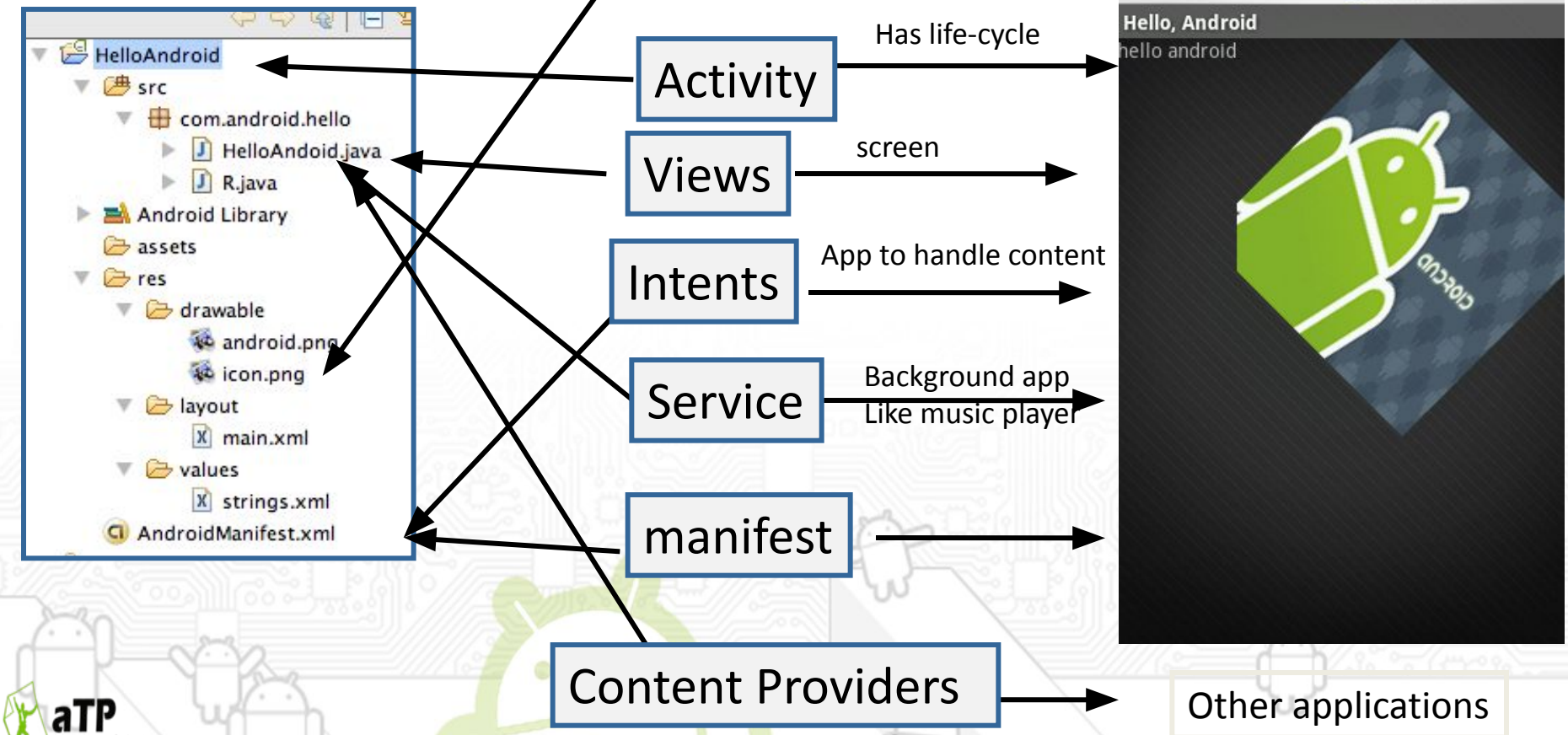
- Android must know that a component of an application exists before it can start it.
- Applications declare their components in an XML manifest file that is always called ***AndroidManifest.xml*** (activities, services, broadcast receivers and content provider)
- The manifest is a structured XML file and is always named AndroidManifest.xml for all applications and that every application must contain.
- The manifest file does many other things
  - Names libraries app needs to be linked against.
  - Identifies permissions the app expects to be granted.

AndroidManifest.xml

Example of a simple manifest file

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.lightcone.webviewdemo"
    android:versionCode="1"
    android:versionName="1.0">
    <application android:icon="@drawable/icon" android:label="@string/app_name">
        <activity android:name=".WebViewDemo" android:label="@string/app_name">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity android:name=".Webscreen" android:label="Web"> </activity>
    </application>
    <uses-sdk android:minSdkVersion="3" />
    <uses-permission android:name="android.permission.INTERNET" />
</manifest>
```

Standard Components form building blocks for Android



Application Context

- Application context refers to the application environment and the process within which all its components are running.

- It allows applications to share the data and resources between various building blocks.

- It is the interface to global information about the application environment.

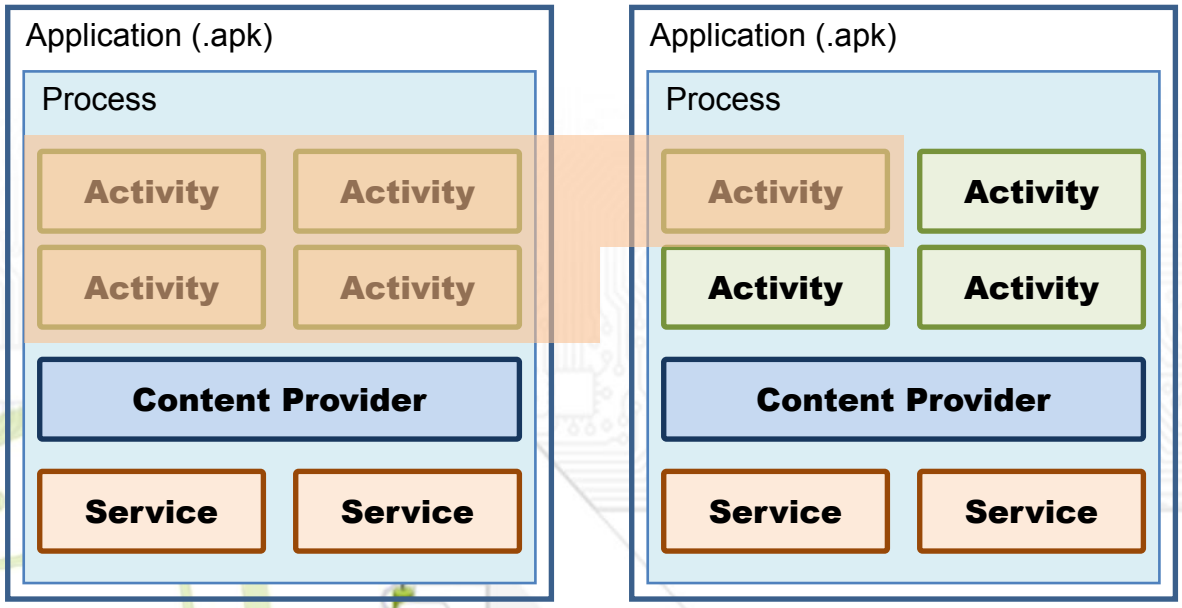
Application Context

- An application context gets created whenever the first component of this application is started up.
- Application context lives as long as your application is alive.
- You can easily obtain a reference to the context by calling *Context.getApplicationContext()* or *Activity.getApplication()*.

Task and Processes

Task

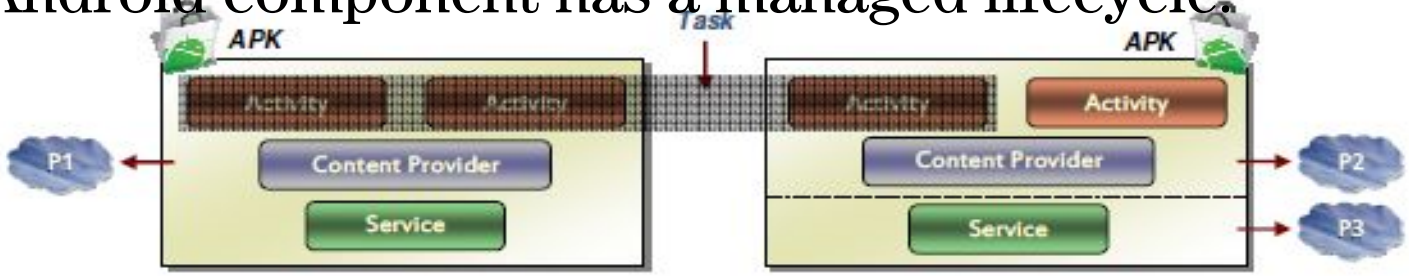
- Different activities can be activated on demand.
- A group of activities composes a *task*. They are maintained in a stack.
- User's experience of an application can actually be a task consisting of different activities from different applications.
- Can have a stack of tasks as well.



Task and Processes

Process

- Android starts a Linux process for each application by default.
- By default, all components of the application run in main thread of application's process.
- Each component can run in its own process.
- Android may decide to kill a process to reclaim resources.
- Each Android component has a managed lifecycle.



Activating Components

- **activities** , **services** , and **broadcast receivers** — are activated by asynchronous messages called ***intents***. that are objects of the *Intent* class.

- Intent messages contains ID of component to be activated + URI of data to be used.

- Content providers are activated when they're targeted by a request from a *ContentResolver*.

Deactivating Components

- Content providers and broadcast receivers are shut down once they are done.
- Activities and services require explicit shutdown.
- Components might also be shut down by the system to reclaim memory for higher priority components (life cycle)

Summary

- **Context -** The context is the central command center for an Android application. All application-specific functionality can be accessed through the context.

- **Activity -** An Android application is a collection of tasks, each of which is called an Activity. Each Activity within an application has a unique task or purpose.

Summary

- **Intent -** The Android operating system uses an asynchronous messaging mechanism to match task requests with the appropriate Activity. Each request is packaged as an Intent. You can think of each such request as a message stating an intent to *do* something.

- **Service -** Tasks that do not require user interaction can be encapsulated in a service. A service is most useful when the operations are lengthy (offloading time-consuming processing) or need to be done regularly (such as checking a server for new mail).

Resources

- http://developer.android.com/guide/topics/fundamentals.html
- http://www.softdevtube.com/2008/09/16/inside-the-android-application-framework/
- http://www.linuxtopia.org/online_books/android/devguide/guide/topics/fundamentals.html